Week 7:
AI Model as a Service I: REST APIs & Deployment Concepts

DS 201: Programming for AI
SPRING 2026
Dr. Mehwish Fatima
Assist Professor | AI & DS
SEECS, NUST

Reading

Designing Machine Learning Systems

- Chapter 6 & 7

Recap

- Week 4: Data Preparation & Representation
- Week 5: ML/DL Programming Pipelines – I
- Week 6: ML/DL Programming Pipelines – II

- This Week: **How trained ML/DL models are deployed as services using REST APIs and Flask for real-time predictions.**

Topics

- Model Deployment
- Model as a Service
- REST Architecture
- FLASK API
- Production considerations

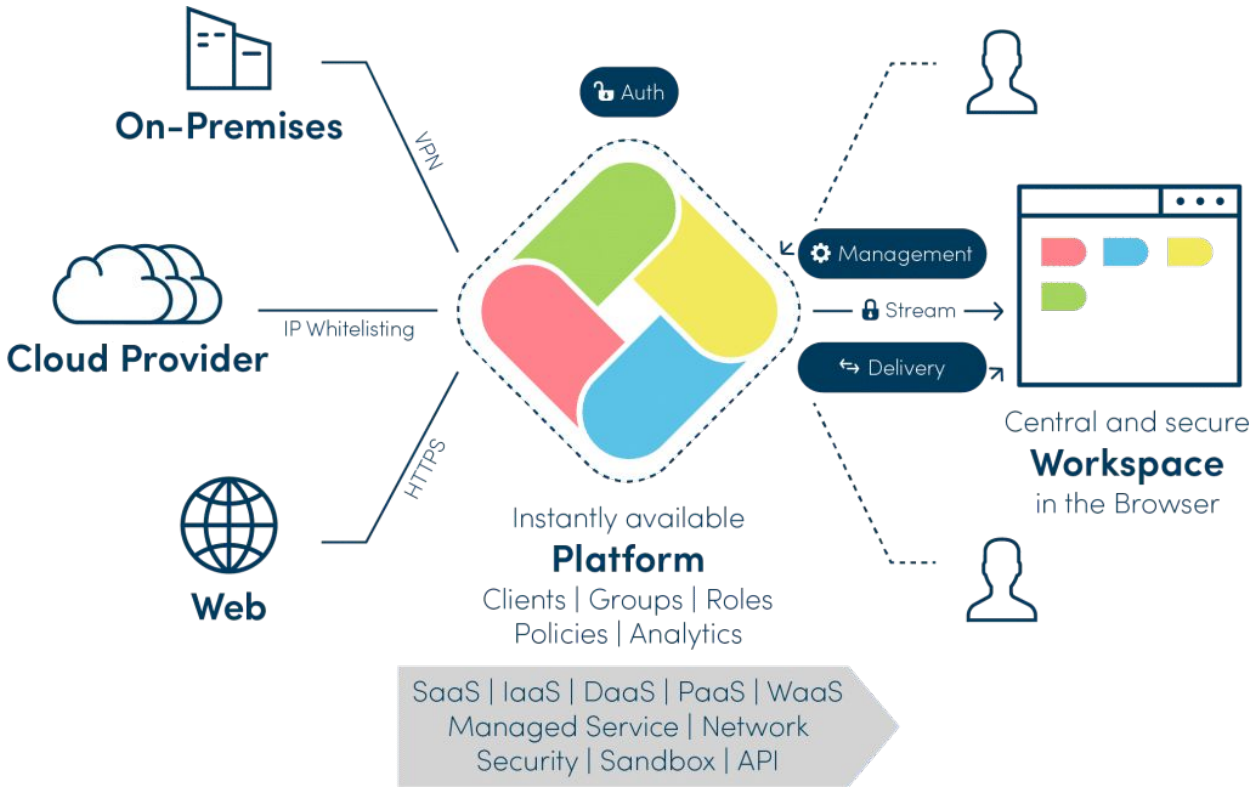
Model Deployment

- Model deployment is the process of integrating a trained machine learning model into a production environment.
- Model can receive inputs and return predictions in real time or batch mode.
- Training is offline experimentation.
- Deployment converts the model into a usable service.

AI Model as a Service (MaaS)

In real-world systems:

- AI models are not used as standalone scripts.
- They are integrated into:
  - Web applications
  - Mobile apps
  - Enterprise systems



AI Model as a Service (MaaS)

- Model as a Service (MaaS) exposes machine learning models via APIs so other applications can consume predictions over HTTP.
- A trained AI model is deployed on a server and accessed via REST APIs.

Example

- Google Translate: User enters text → app sends request to API → AI model translates → result is returned instantly.

Why Using REST for AI Models?

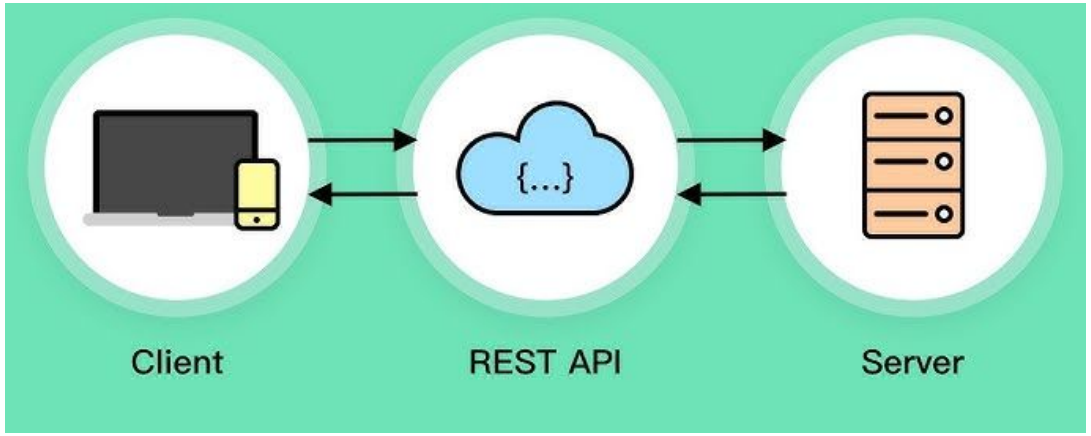
"Why not just run the Python file?"

- Applications need network communication
- Enables models to be accessed by web and mobile applications
- Simple and widely supported communication method
- Scalable and easy to integrate into real-world systems
- Allows real-time predictions

REST Architecture

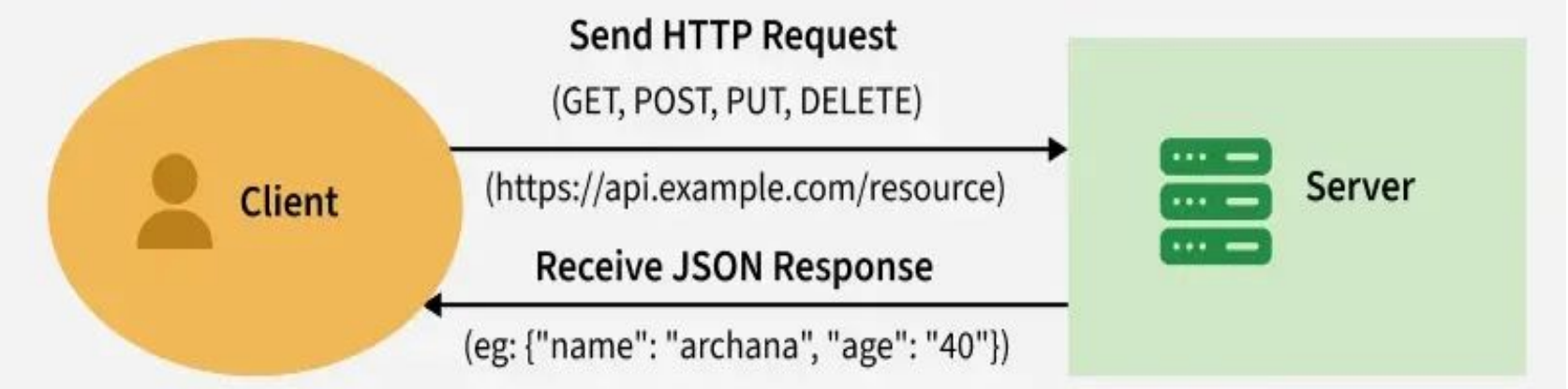
REST

- REST (Representational State Transfer) is an architectural style used to build network-based applications.
- It allows different systems to communicate over the internet using standard HTTP protocols.
-



https://www.w3schools.com/nodejs/nodejs_rest_api.asp

REST API



- REST APIs work by sending requests and receiving responses, typically in JSON format, between the client and server.
  - A request is sent from the client to the server via a web URL, using one of the HTTP methods.
  - The server then responds with the requested resource, which could be HTML, XML, Image, or JSON, with JSON being the most commonly used format for modern web services.
  - These methods map to **CRUD (Create, Read, Update, Delete) operations** for managing resources on the web.

REST API: Core Elements

REST API relies on the following three elements:

- **Client**
  - The client is the software code or application that requests a resource from a server.

- **Server**
  - The server is the software code or application that controls the resource and responds to client requests for the resource.

- **Resource**
  - The resource is any data or content that the server controls and makes available in response to client requests.

REST API: Core Elements

To access a resource, the client sends an HTTP request to the server. Client requests include the following four parts:

- **HTTP methods**
  - **This shows what should happen to the specified resource.**
  - The four fundamental HTTP methods are known **as verbs**.
  - **POST** creates a new resource
  - **GET** retrieves an existing resource
  - **PUT** updates or changes an existing resource, and
  - **DELETE** deletes a resource.

REST API: HTTP Methods

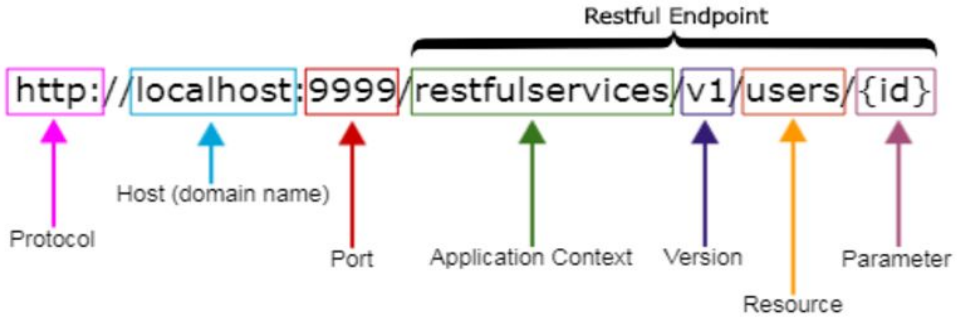
Method	Purpose	Example
GET	Retrieve data	Get prediction history
POST	Send data	Send features for prediction
PUT	Update data	Update model version
DELETE	Remove data	Delete logs

https://www.geeksforgeeks.org/node-js/rest-api-introduction/

HTTP Status Codes

Code	Meaning
200	Success
201	Created
400	Bad request
401	Unauthorized
404	Not found
500	Server error

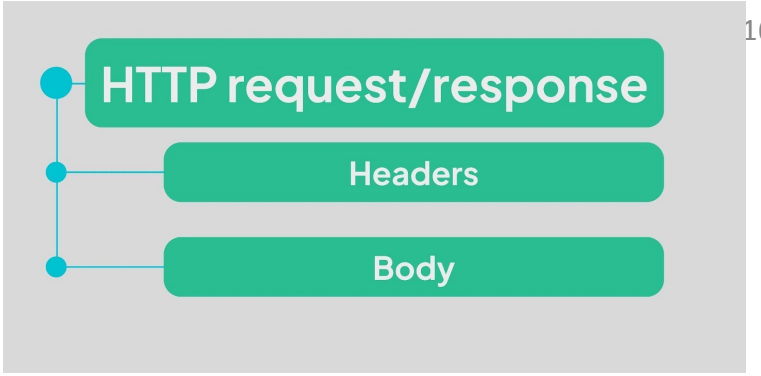
REST API: Core Elements



To access a resource, the client sends an HTTP request to the server. Client requests include the following four parts:

- **Endpoint**
  - The endpoint shows where the resource is located and typically includes a **Uniform Resource Identifier (URI)**.
  - If the resource is accessed through the internet, the URI can be a URL that provides a web address for the resource.

REST API: Core Elements



To access a resource, the client sends an HTTP request to the server. Client requests include the following four parts:

- **Header**
  - A header includes the details needed to execute the call and handle the response.
  - **A request header** might include:
    - authentication data,
    - an encryption key,
    - more details about the server location or access information and
    - details about the desired data format needed for the response.

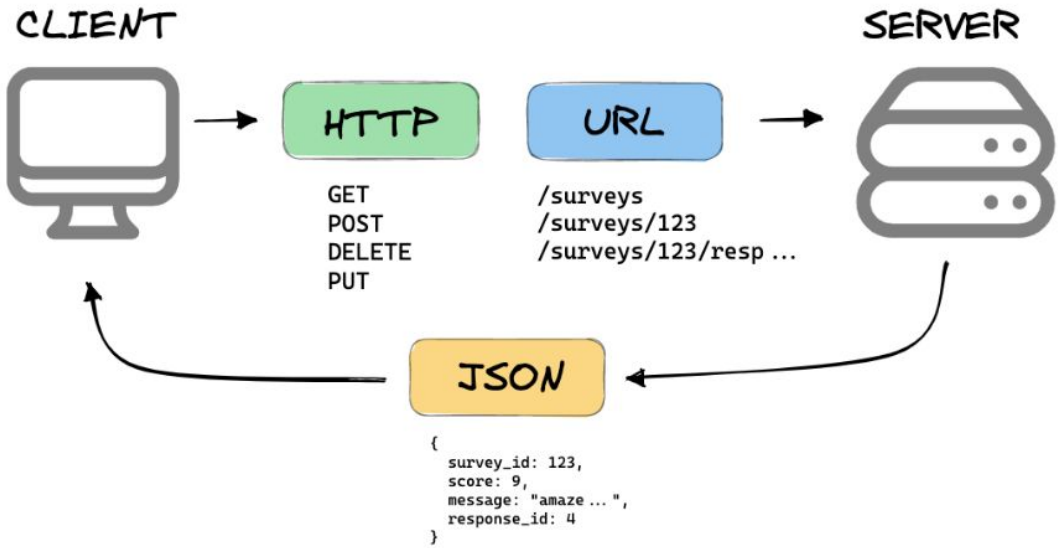
REST API: Core Elements

To access a resource, the client sends an HTTP request to the server. Client requests include the following four parts:

- **Body**
  - The body contains relevant information to or from the server.
  - For example, a body might contain the new data to be added to the server through a POST or PUT method.

How REST Works in AI Deployment?

- Client sends input data to the API
- API forwards data to the model
- Model processes the input
- API returns prediction response



Flask

Flask

Flask is a lightweight and versatile micro-framework for Python for web development.

Advantages:

- **Lightweight and Minimalistic**
  - Flask is lightweight and follows a minimalistic design philosophy, allowing developers to build web applications without unnecessary overhead.
- **Flexibility**
  - Flask provides a flexible and modular structure, allowing developers to choose the components and extensions that best suit their project requirements.
- **Easy to Learn and Use**
  - Flask has a simple and intuitive syntax, making it easy for beginners to learn and use.
  - It provides clear documentation and a supportive community for assistance.

Flask

Flask is a lightweight and versatile micro-framework for Python for web development.

Advantages:

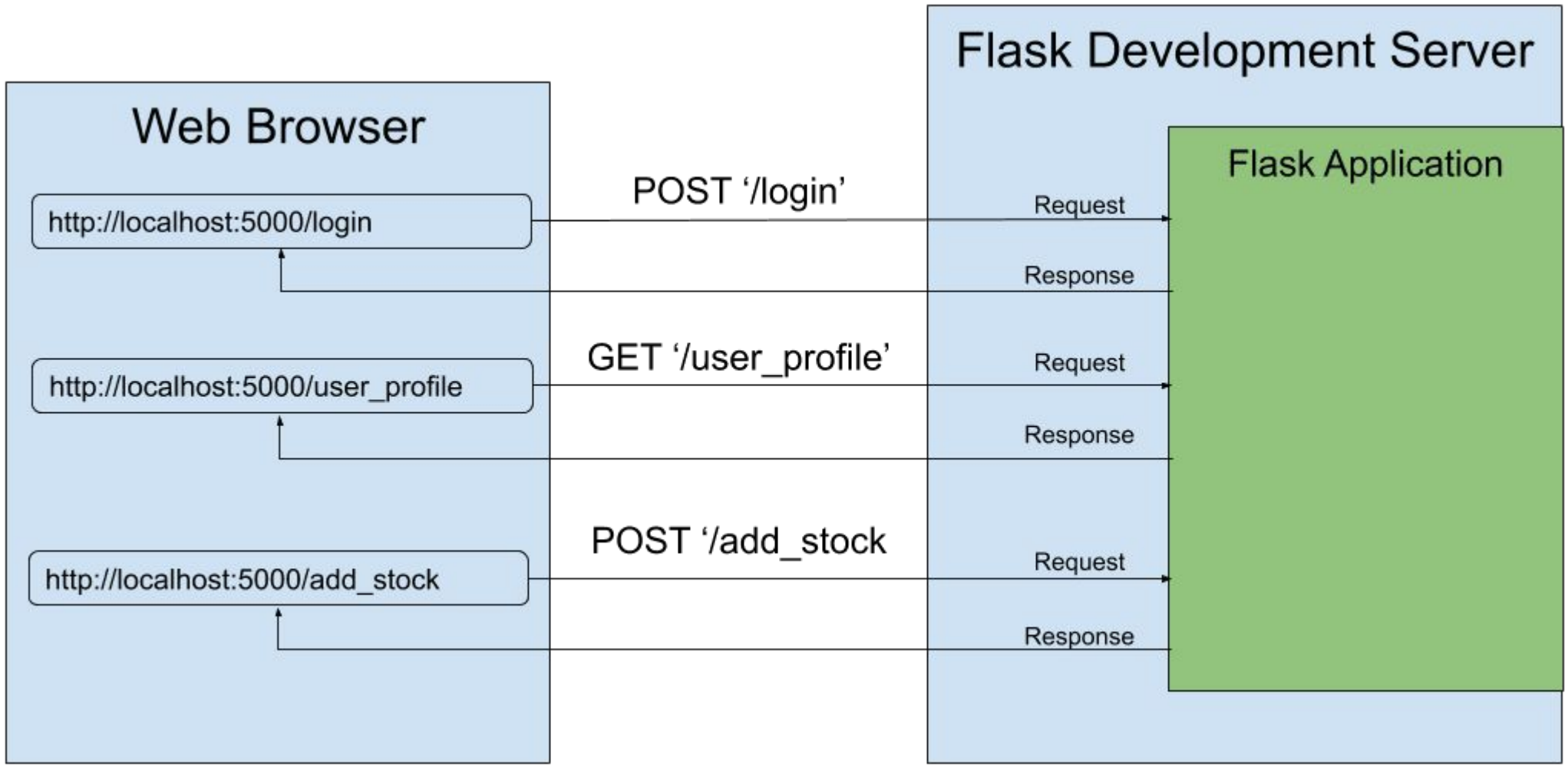
- **Extensible**
  - Flask offers a wide range of extensions and plugins that enhance its functionality, including support for databases, authentication, and RESTful APIs.
- **Jinja2 Templating**
  - Flask integrates seamlessly with Jinja2, a powerful and feature-rich templating engine, allowing developers to create dynamic and reusable HTML templates.
- **Built-in Development Server**
  - Flask comes with a built-in development server, making it easy to test and debug web applications during the development process.

Flask

Flask is a lightweight and versatile micro-framework for Python for web development.

Advantages:

- **Scalability**
  - Flask is scalable and can be used to build both small-scale and large-scale web applications. It supports various deployment options, including traditional servers, cloud platforms, and containerization.


Flask Development Server
Web Browser
Flask Application
http://localhost:5000/login
POST '/login'
Request
Response
http://localhost:5000/user_profile
GET '/user_profile'
Request
Response
http://localhost:5000/add_stock
POST '/add_stock
Request
Response

Code Walk Through

https://colab.research.google.com/drive/12BMufEh6dbPTp8MbpZ5IIEMyhrVWqsWE?usp=sharing

```python
from flask import Flask, request, jsonify
from threading import Thread
import requests

app = Flask(__name__)

# Dummy model
def predict(text):
    return {"length": len(text), "uppercase": text.upper()}

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200


@app.route("/models", methods=["GET"])
def list_models():
    return jsonify({"models": ["dummy-model"]}), 200


@app.route("/predict", methods=["POST"])
def predict_endpoint():
    if not request.is_json:
        return jsonify({"error": "Invalid JSON"}), 400
    if "text" not in data:
        return jsonify({"error": "Missing 'text'"}), 400

    result = predict(data["text"])
    return jsonify(result), 200
def run_app():
    app.run(port=5001)

thread = Thread(target=run_app)
thread.daemon = True
thread.start()
```

Code Walk Through

- You are running:
  - A Flask web server
  - Inside a Jupyter notebook
  - In a background thread
  - Listening on localhost:5001

- So the notebook is acting as:
  - Server (Flask app)
  - Client (requests)
  - All on the same machine
- This simulates a production inference server, just locally.

Imports

- **Flask**
  - Creates the web application object.

- **request**
  - Represents the incoming HTTP request:
    - headers
    - JSON body
    - method
    - URL

- **jsonify**
  - Converts Python dict → proper JSON response
  - Also sets Content-Type: application/json.

```
from flask import Flask, request, jsonify
from threading import Thread
import requests
```

Imports

- **Thread**
  - Needed because Jupyter blocks execution.
  - Flask runs in a background thread.

- **requests**
  - Acts as a client inside notebook to test our API.

```
from flask import Flask, request, jsonify
from threading import Thread
import requests
```

Create Application Instance

```
app = Flask(__name__)
```

- This creates the WSGI application object.
  - **WSGI (Web Server Gateway Interface)** is a standard interface between:
    - a Python web application
    - and a web server
  - It defines how HTTP requests are passed to Python code and how responses are returned.

- **app** is the server
  - It registers routes
  - It handles incoming HTTP calls

Dummy Model (Simulating ML Inference)

- **This simulates a deployed ML model**.
  - a model inference function.
- This function:
  - compute string length
  - convert to uppercase

```python
# Dummy model
def predict(text):
    return {"length": len(text), "uppercase": text.upper()}
```

- This isolates model logic from API logic → clean separation of concerns.

- Architecture separation:

Route → Validation → Model → JSON Response

Health Endpoint

```python
@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200
```

- @app.route(...) registers a URL
- /health is the resource
- methods=["GET"] restricts allowed verbs
- When **client calls**:
  - GET /health

- **Server**:
  - Matches route
  - Executes health()
  - Returns JSON + HTTP status code 200

Property	Meaning in /health
GET	Retrieval-only operation
Stateless	No server memory dependency
200	Successful synchronous response

Health Endpoint

```python
@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200
```

- In production:
  - load balancers
  - Kubernetes liveness/readiness probes
  - monitoring systems

- If it returns anything other than 200:
  - Traffic may be stopped
  - Pod may be restarted

Property	Meaning in /health
GET	Retrieval-only operation
Stateless	No server memory dependency
200	Successful synchronous response

Models Endpoint

```python
@app.route("/models", methods=["GET"])
def list_models():
    return jsonify({"models": ["dummy-model"]}), 200
```

- **Expose available models.**
- In real systems:
  - multiple deployed models
  - versioning (v1, v2)
  - model registry integration

- Design principle:
  - Resources should be nouns (/models), not verbs.

Idempotency

- A request is idempotent if:
  - Repeating the same request multiple times produces the same system state.

- Examples:
  - GET → idempotent

  - DELETE → idempotent

  - POST → usually not

Predict Endpoint

- We send data in request body.
- It changes computational state (non-idempotent).
- **Why POST?**
  - GET should not contain payload.

- **Validate JSON**
  - Prevents:
    - malformed input
    - incorrect content type
  - Returns:
    - 400 Bad Request

```python
@app.route("/predict", methods=["POST"])
def predict_endpoint():
    if not request.is_json:
        return jsonify({"error": "Invalid JSON"}), 400

    data = request.get_json()

    if "text" not in data:
        return jsonify({"error": "Missing 'text'"}), 400

    result = predict(data["text"])
    return jsonify(result), 200
```

Predict Endpoint

```python
@app.route("/predict", methods=["POST"])
def predict_endpoint():
    if not request.is_json:
        return jsonify({"error": "Invalid JSON"}), 400

    data = request.get_json()

    if "text" not in data:
        return jsonify({"error": "Missing 'text'"}), 400

    result = predict(data["text"])
    return jsonify(result), 200
```

- **Extract Data**
  - Parses incoming JSON → Python dictionary.
- **Field Validation**
  - Defensive programming:
    - Never trust client input.
- **Call Model**
  - Important architectural principle:
    - API layer does NOT contain model logic.
    - It delegates.
- **Return Response**
  - Response lifecycle:
    - Client → HTTP POST → Flask route → Model → JSON response → Status code

Running Flask in Jupyter

- Flask normally blocks execution.
  - Internally it:
    - Opens a socket
    - Enters an infinite loop
    - Waits for incoming HTTP requests
- Jupyter would freeze.

```
def run_app():
    app.run(port=5001)

thread = Thread(target=run_app)
thread.daemon = True
thread.start()
```

- So we run it in a background thread:
- Why daemon?
  - Thread exits automatically when notebook kernel stops.

Running Flask in Jupyter

- Flask normally blocks execution.
  - Internally it:
    - Opens a socket
    - Enters an infinite loop
    - Waits for incoming HTTP requests

- Jupyter would freeze.
  - while True:
  - wait_for_request()
  - handle_request()

```
def run_app():
    app.run(port=5001)

thread = Thread(target=run_app)
thread.daemon = True
thread.start()
```

Running Flask in Jupyter

```
def run_app():
    app.run(port=5001)

thread = Thread(target=run_app)
thread.daemon = True
thread.start()
```

- Jupyter would freeze.
  - while True:
  - wait_for_request()
  - handle_request()

- This l**oop never returns control** to the caller.
  - In a normal .py file → fine.
  - In Jupyter → problem.

- Jupyter executes cell → waits for completion.
- If Flask blocks → the cell never finishes → notebook appears frozen.

Running Flask in Jupyter

- Using a Background Thread
- Now execution splits:
  - **Main thread** → continues running notebook
  - **Background thread** → runs Flask server loop

- Main thread = interactive notebook
- Worker thread = web server

- They run **concurrently**.

```
def run_app():
    app.run(port=5001)

thread = Thread(target=run_app)
thread.daemon = True
thread.start()
```

Running Flask in Jupyter

```
def run_app():
    app.run(port=5001)

thread = Thread(target=run_app)
thread.daemon = True
thread.start()
```

- Threads in Python are of two types:

Type	Behavior
Non-daemon	Must finish before program exits
Daemon	Automatically killed when main program exits

- If we don’t set daemon:
  - Flask thread keeps running
  - Notebook kernel cannot shut down cleanly
  - You get hanging processes

Running Flask in Jupyter

```
def run_app():
    app.run(port=5001)

thread = Thread(target=run_app)
thread.daemon = True
thread.start()
```

- Threads in Python are of two types:

Type	Behavior
Non-daemon	Must finish before program exits
Daemon	Automatically killed when main program exits

- With daemon:
  - When kernel stops → main thread ends
  - Python automatically kills daemon threads
  - No cleanup headache

Running Flask in Jupyter

- Flask dev server:
  - Stateless
  - No file writes
  - No DB transactions
  - So abrupt termination is acceptable.

- In production?
  - Use Gunicorn (traditional synchronous WSGI)/ Uvicorn (modern asynchronous ASGI)
  - Handle graceful shutdown
  - Close DB connections
  - Flush logs

- **Daemon threads are not production-grade lifecycle management.**

How Requests Are Handled

- Flask dev server: single-threaded, blocking
- Each request occupies the server until response is returned

- Under load → queue forms → latency increases

- Production servers use worker processes / async handling

System Architecture View

Client (requests)

↓

Flask Router

↓

Endpoint Function

↓

Model Function

↓

JSON Response

- This mirrors real LLM serving pipelines:
  - FastAPI → inference wrapper → model → response serialization.

REST Properties Demonstrated

Property	Where Shown
Statelessness	No session memory
Resource-based URLs	/models, /predict
HTTP methods	GET vs POST
Proper status codes	200, 400
JSON representation	jsonify()

From Prototype to Production: What's Missing?

- **"This works. But this is not production."**

- Authentication
- Logging
- Validation
- Persistence
- Scaling

Reflections

- Is /predict idempotent?
- Could /predict be GET?
- What breaks under concurrent load?
- Where would GPU model loading happen?
- What if input text is 10MB?

From Server to Client

```
# Test GET
requests.get("http://127.0.0.1:5001/models").json()
```

- Earlier, we explained the server side:
  - Flask app created
  - /models route defined
  - list_models() returns JSON + 200

- Now we are at client side:
  - Previously we built the endpoint.
  - Now we are behaving like an external client calling it.

Closing Notes

- Trained models become useful only when exposed as services
- REST enables standardized network-based communication
- APIs follow resource-oriented design
- HTTP methods define behavior
- Status codes define outcome
- Flask allows rapid prototyping of inference services
- Client and server are independent systems communicating over HTTP
- A working API is not the same as a production-ready system

Any Questions?